

Московский институт радиотехники, электроники и автоматики
(технический университет)

**Практическое применение
алгоритма шифрования ГОСТ 28147-89**

Автор: Горинов И.А.

Москва 2000

Содержание

Введение	3
2. Описание алгоритма	4
2.1. Режим простой замены	4
2.2. Режим гаммирования	5
2.3. Режим гаммирования с обратной связью	6
3. Ключевые элементы	7
3.1. Блок подстановки	7
3.2. Ключи шифрования	10
4. Особенности программной реализации	11
4.1. Блок подстановки	11
4.2. Гаммирование	12
Литература	13
Исходный текст функции зашифрования в режиме простой замены	14
Исходный текст функции шифрования в режиме гаммирования	15
Аннотация	17

Введение

Данная работа посвящена алгоритму шифрования, созданному с целью установить единый алгоритм криптографического преобразования для вычислительных машин, комплексов и сетей. Сначала этот алгоритм был засекречен, затем уровень секретности была понижена до ДСП. Сейчас алгоритм является открытым и широко применяется как в России, так и в других странах, но некоторые детали остались нераскрытыми, например, требования к таблицам замен. Шифр создавался в 70-е годы, когда даже не предполагали, какого уровня достигнет вычислительная техника сегодня, но тем не менее в нем заложен большой запас прочности, и сейчас он обеспечивает высокий уровень надежности.

В качестве примеров реализации используется текст на языке ассемблера i386. Языки высокого уровня могут использоваться для описания и изучения алгоритма, но для практической реализации неприменимы: требуется максимальное быстродействие, а алгоритм содержит большое число битовых операций. Для примеров используется код Intel 386. Это самый распространенный набор команд, поддерживаемый всеми современными процессорами Intel и их аналогами. Для других 32-разрядных процессоров основные принципы реализации алгоритма те же, что и для i386.

2. Описание алгоритма

2.1. Режим простой замены

В режиме простой замены открытые данные разбивают на блоки размером 64 бита. Алгоритм зашифрования 64-разрядного блока состоит из 32 циклов, в каждом из которых используется одна 32-битная часть ключа $X_0, \dots, X_7$. При этом части ключа используются в следующей последовательности:

$$\begin{array}{cccccccc} X_0, & X_1, & X_2, & X_3, & X_4, & X_5, & X_6, & X_7, & X_0, & X_1, & X_2, & X_3, & X_4, & X_5, & X_6, & X_7, \\ X_0, & X_1, & X_2, & X_3, & X_4, & X_5, & X_6, & X_7, & X_7, & X_6, & X_5, & X_4, & X_3, & X_2, & X_1, & X_0. \end{array}$$

Цикл шифрования заключается в следующем:

1) 64-битный блок данных разбивается на 2 32-битных части: $N1$ и $N2$.

2) $N1$ складывается по модулю 2^{32} с соответствующей частью ключа X_j .

3) сумма пропускается через блок подстановки, состоящий из 8 узлов замены $K_0 \dots K_7$.

Поступающий на блок подстановки 32-разрядный вектор разбивается на 8 последовательных 4-разрядных векторов, каждый из которых преобразуется в 4-разрядный вектор соответствующим узлом замены.

Узел замены представляет собой таблицу из 16 строк, содержащих по 4 бита заполнения в строке. Входной вектор определяет адрес строки в таблице, заполнение данной строки является выходным вектором (о заполнении таблиц - см. 2.1).

Затем 4-разрядные выходные векторы последовательно объединяются в 32-битное значение.

4) Результат циклически сдвигается влево (в сторону старших разрядов) на 11 бит.

5) Результат сдвига складывается по модулю 2 с $N2$.

6) $N1$ и $N2$ меняются местами, если это не последний цикл.

Расшифрование осуществляется по тому же алгоритму, но части ключа используются в обратном порядке:

$$\begin{array}{cccccccc} X_7, & X_6, & X_5, & X_4, & X_3, & X_2, & X_1, & X_0, & X_7, & X_6, & X_5, & X_4, & X_3, & X_2, & X_1, & X_0, \\ X_7, & X_6, & X_5, & X_4, & X_3, & X_2, & X_1, & X_0, & X_0, & X_1, & X_2, & X_3, & X_4, & X_5, & X_6, & X_7. \end{array}$$

2.2. Режим гаммирования

В режиме гаммирования открытые данные, разбитые на 64-разрядные блоки, зашифровываются путем поразрядного сложения по модулю 2 с гаммой шифра $\Gamma_{\text{ш}}$, которая вырабатывается блоками по 64 бита. Число разрядов в блоке $T_o^{(M)}$ может быть меньше 64.

$$\Gamma_{\text{ш}} = (\Gamma_{\text{ш}}^{(1)}, \Gamma_{\text{ш}}^{(2)}, \dots, \Gamma_{\text{ш}}^{(M-1)}, \Gamma_{\text{ш}}^{(M)}).$$

Для получения гаммы используется генератор числовой последовательности с периодом $2^{32}(2^{32}-1)$, которая зашифровывается в режиме простой замены. Для инициализации генератора используется синхропосылка:

$$\begin{aligned} (Y_0, Z_0) &= A(S); \\ T_{\text{ш}}^{(i)} &= A((Y_{i-1} + C_2) \bmod 2^{32}, (Z_{i-1} + C_1) \bmod^* 2^{32}-1) \oplus T_o^{(i)} = \Gamma_{\text{ш}}^{(i)} \oplus T_o^{(i)}, \\ i &= 1 \dots M, \end{aligned}$$

(A - функция шифрования в режиме простой замены).

При расшифровании зашифрованных данных используется тот же алгоритм, что и при зашифровании:

$$\begin{aligned} T_o^{(i)} &= A((Y_{i-1} + C_2) \bmod 2^{32}, (Z_{i-1} + C_1) \bmod^* 2^{32}-1) \oplus T_{\text{ш}}^{(i)} = \Gamma_{\text{ш}}^{(i)} \oplus T_{\text{ш}}^{(i)}, \\ i &= 1 \dots M. \end{aligned}$$

“ $X \bmod^* 2^{32}-1$ ” - операция, подобная сложению по модулю $2^{32}-1$, с одним исключением - вместо результата 0 получается $2^{32}-1$. Подробнее эта операция рассмотрена в 3.2.

2.3. Режим гаммирования с обратной связью

В режиме гаммирования с обратной связью открытые данные, разбитые на 64-разрядные блоки, зашифровываются путем поразрядного сложения по модулю 2 с гаммой шифра $\Gamma_{\text{ш}}$. Число разрядов в блоке $T_o^{(M)}$ может быть меньше 64. При этом первый блок гаммы вырабатывается путем зашифрования синхропосылки в режиме простой замены, а последующие - путем зашифрования предыдущего блока зашифрованных данных в режиме простой замены:

$$\begin{aligned} T_{\text{ш}}^{(1)} &= A(S) \oplus T_o^{(1)} = \Gamma_{\text{ш}} \oplus T_o^{(1)} \\ T_{\text{ш}}^{(i)} &= A(T_{\text{ш}}^{(i-1)}) \oplus T_o^{(i)} = \Gamma_{\text{ш}}^{(i)} \oplus T_o^{(i)}, \quad i = 2 \dots M. \end{aligned}$$

(A - функция шифрования в режиме простой замены).

Для расшифрования используется тот же алгоритм. Первый блок гаммы вырабатывается путем зашифрования синхропосылки в режиме простой замены, а последующие - путем зашифрования предыдущего блока зашифрованных данных в режиме простой замены:

$$\begin{aligned} T_o^{(1)} &= A(S) \oplus T_{\text{ш}}^{(1)} = \Gamma_{\text{ш}} \oplus T_{\text{ш}}^{(1)} \\ T_o^{(i)} &= A(T_{\text{ш}}^{(i-1)}) \oplus T_{\text{ш}}^{(i)} = \Gamma_{\text{ш}}^{(i)} \oplus T_{\text{ш}}^{(i)}, \quad i = 2 \dots M. \end{aligned}$$

В этом режиме гамма шифра зависит не только от синхропосылки, но и от шифруемых данных. Режим гаммирования с обратной связью, в отличие от остальных, обеспечивает некоторую аутентичность: если один из зашифрованных блоков был изменен, то этот и последующие блоки будут расшифрованы неправильно. Для защиты от навязывания ложных данных существует режим выработки имитовставки, основанный на упрощенном алгоритме зашифрования в режиме простой замены [1].

3. Ключевые элементы

3.1. Блок подстановки

Блок подстановки состоит из 8 узлов замены. В качестве узлов замены в алгоритме используются 4-битные таблицы. В отличие от DES [2], эти таблицы не являются фиксированными, они являются ключевыми элементами, общими для сети ЭВМ. В стандарте указано только одно требование к ним - каждое из чисел 0 .. 15 встречается только один раз. При наличии повторяющихся элементов обратимость алгоритма сохраняется, но снижается криптографическая стойкость (подробнее об этом в [5,6]). и не указаны другие требования к ним. Возможно использование таблиц в виде случайных перестановок чисел 0 .. 15 (число возможных перестановок - 16!), но в этом случае существует вероятность попадания на криптографически слабые варианты перестановок. Примеры таких вариантов - отсутствие замены, когда значение каждого бита на выходе равно значению на входе [6], или блоки коммутации, в которых биты просто меняются местами [4]. Рассмотрим такую таблицу:

X	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
y_0	1	0	1	0	0	0	1	1	1	1	0	0	0	1	1	0
y_1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
y_2	0	1	1	0	1	0	0	1	1	1	1	1	0	0	0	0
y_3	1	0	1	1	1	0	1	0	1	0	1	0	1	0	0	0

На первый взгляд перестановка выглядит достаточно случайной, но она оставляет неизменным бит 1 при любой комбинации остальных битов. Это означает, что блок подстановки на самом деле 3-битный, а один бит проходит через него без изменений. Еще одна таблица:

X	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
y_0	1	0	1	1	1	1	0	0	0	0	0	1	1	0	0	1
y_1	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0
y_2	1	1	0	1	0	1	1	0	1	0	0	0	0	0	1	1
y_3	0	0	1	1	1	0	0	0	1	0	1	0	0	1	1	1

Бит 1 на выходе блока всегда равен инвертированному биту 2 на входе, независимо от значения остальных входных битов. Это такая же сильная связь, как и в предыдущем примере.

Из этого можно сформулировать дополнительное требование к блокам подстановки: каждый бит на выходе блока должен быть статистически независим от каждого бита на входе. Допустим, что значения битов на входе (факторные признаки) - независимые случайные величины с равномерным распределением, т.е. с равной вероятностью принимают значения 0 и 1. Тогда значения бит на выходе (результативные признаки) тоже будут иметь равномерное распределение. Для оценки таблицы можно построить матрицу коэффициентов корреляции, каждый элемент p_{ij} которой содержит коэффициент корреляции между битом i числа k на входе блока и битом j числа $s(k)$ на выходе:

$$p_{ij} = 1 - \frac{\sum_k k_i \oplus s(k)_j}{2^{N-1}}$$

При этом возможны следующие варианты: если $p = 1$, то значение бита j на выходе равно значению бита i на входе при любых комбинациях бит на входе. Если $p = -1$, значит, значение бита j на выходе всегда является инверсией входного бита i , что является такой же сильной зависимостью. Если $p = 0$, то выходной бит j с равной вероятностью принимает значения 0 и 1 при любом фиксированном значении входного бита i . Можно сделать предположение, что максимально стойкими будут блоки, в которых парная корреляция будет наименьшей, т.е. все коэффициенты будут иметь значение 0. Для таких таблиц значения входного и выходного 4-битных чисел будут иметь функциональную связь, но коэффициент корреляции между ними будет тоже равен 0. Пример узла замены с нулевыми коэффициентами корреляции:

X	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
y₀	1	1	0	0	1	0	1	0	0	0	1	1	0	1	0	1
y₁	0	1	0	0	1	1	0	1	1	0	1	1	0	0	1	0
y₂	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
y₃	1	0	0	0	0	1	1	1	0	1	1	1	1	0	0	0

Если часть входных бит имеет фиксированные значения, а значения остальных - случайные величины с равномерным распределением, то каждый бит на выходе тоже будет случайной величиной с равномерным распределением.

Подобные блоки подстановки могут быть получены путем перебора случайно сгенерированных таблиц.

Таблицы, используемые в различных реализациях алгоритма (ARJ 2.60, BestCrypt), обычно не удовлетворяют этому условию, но сильной связи между битами нет. Таким же свойством обладают блоки подстановки американского стандарта DES, имеющие 6 бит на входе и 4 бита на выходе [2]. Возможно, что все эти таблицы построены случайным образом без использования каких-то проверок.

Рассмотрим один из узлов замены, использующийся в модуле шифрования архиватора ARJ 2.60 (arjcrypt.com):

X	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
y_0	1	1	0	1	1	1	1	1	0	0	0	1	0	0	0	0
y_1	0	1	0	0	1	1	0	0	0	1	1	1	1	0	1	0
y_2	1	0	1	0	0	1	1	0	0	0	1	1	1	0	0	1
y_3	1	1	0	0	0	1	0	1	0	1	1	0	0	1	0	1

Матрица коэффициентов корреляции для этого узла (каждая строка соответствует входному биту, столбец - выходному):

	0	1	2	3
0	-1/4	0	1/4	-1/2
1	0	1/4	-1/4	1/4
2	0	0	0	0
3	3/4	-1/4	0	0

Коэффициент корреляции между входным битом 3 и выходным битом 0 равен 3/4.

Их значения равны в 14 комбинациях из 16 (87.5%), что является сильной зависимостью. Такая же сильная связь наблюдается в 2 таблицах подстановки программы BestCrypt. Подобные узлы замен могут существенно снизить устойчивость шифра к криптоанализу.

3.2. Ключи шифрования

На ключи шифрования сложнее определить какие-либо требования, поскольку обычно это пароль, т.е. строка символов, серийный номер устройства (процессор, дисковый накопитель, ключ аппаратной защиты). При правильно построенных блоках подстановки эти требования и не нужны - даже при нулевом ключе исходные данные будут искажены достаточно сильно. Но если есть возможность, лучше всего использовать в качестве ключевых случайные данные. Такой подход можно применить, когда ключевые записываются на внешний носитель - например, в ключ аппаратной защиты или смарт-карту. При этом в качестве генератора случайных чисел можно использовать гамму шифра в режиме гаммирования (2.2).

Если в качестве ключа используется пароль из 8-битных символов, то старший бит обычно имеет фиксированное значение, при использовании 16-битных символов Unicode половина байтов ключа будет иметь одно и то же значение (если символы из одного алфавита). Чтобы этого не происходило, нужно увеличить длину пароля, а в качестве ключа использовать некоторую функцию от символов пароля. Например, для 8-битных символов можно увеличить длину с 32 до 36 байт и использовать только младшие 7 разрядов кодов символов. Для 16-битных символов Unicode можно отбросить байт, определяющий номер алфавита.

4. Особенности программной реализации

4.1. Блок подстановки

Одна из самых медленных операций в алгоритме - это операция замены 4-битных блоков. С помощью операций сдвига из 32-разрядного регистра выделяются части по 4 бита, которые используются в качестве индексов в таблице замен. С помощью простых битовых операций можно преобразовать 4-битные узлы замен в 8-битные. Расширение блоков подстановки до 8 бит позволяет не только в 2 раза уменьшить количество таких сдвигов. В этом случае операция замены сразу 2 блоков производится одной командой процессора (XLAT), которая заменяет значение регистра AL (младшие 8 бит 32-битного регистра EAX) соответствующим значением из таблицы:

```
xlat
add    ebx, 100h
ror    eax, 8
xlat
add    ebx, 100h
ror    eax, 8
xlat
add    ebx, 100h
ror    eax, 8
xlat
add    ebx, 100h
rol    eax, 3
```

Последний циклический сдвиг вправо на 8 бит вместе с последующим сдвигом влево на 11 заменены одной операцией сдвига влево на 3 бита, что дает дополнительную оптимизацию.

В результате размер таблицы увеличивается до 256 байт, а весь блок подстановки - до 4Kb, что намного меньше размера кэша современных процессоров.

Расширение до 16 бит уже не имеет смысла: замена 16-битных групп требует дополнительных битовых операций, а общий размер блока подстановки увеличивается до 256Kb.

Иногда утверждают, что для достижения максимальной производительности нужно отказаться от вложенных циклов и использовать вместо них повторяющиеся участки кода. Это верно только для процессоров с конвейерной обработкой команд без предсказания ветвлений, таких, как 386, 486. Для остальных быстродействие может только снизиться, т.к. короткие часто повторяющиеся циклы попадают в кэш, а длинный участок кода придется выбирать из внешней памяти.

4.2. Гаммирование

В режиме гаммирования используется генератор числовой последовательности с большим периодом. Генератор последовательности состоит из двух независимых генераторов с взаимно простыми периодами. Сложение по модулю 2^{32} реализуется очень просто - на 32-разрядных процессорах достаточно выполнить операцию сложения, игнорируя перенос:

```
add     eax, _C2
```

Операция, которая определена в алгоритме как “суммирование по модулю $2^{32}-1$ ”, на самом деле представляет собой другую похожую операцию, которую проще реализовать на практике, не используя медленных операций деления:

$a [+]' b = a + b$, если $a + b < 2^{32}$;
 $a [+]' b = a + b - 2^{32} + 1$, если $a + b \geq 2^{32}$.

Результат этой операции равен сумме по модулю $2^{32}-1$, кроме случая, когда $a + b = 2^{32}-1$. В этом случае при суммировании по модулю должен получиться 0, но данная операция дает результат $2^{32}-1$. Такая операция реализуется просто - достаточно сложить два числа и к сумме добавить перенос:

```
add     eax, _C1  
adc     eax, 0
```

Аппаратная реализация еще проще - выход переноса C_{n+1} 32-разрядного сумматора замыкается на вход переноса C_n .

Значения констант C1, C2 указаны в стандарте:

```
C1 = 000000001000000001000000001000001002  
C2 = 000000001000000001000000001000000012
```

Период первого генератора равен 2^{32} , второго - $2^{32}-1$. Поскольку эти числа взаимно простые, период смешанной последовательности будет равен их произведению, что вполне достаточно для шифрования больших объемов данных. Сама по себе последовательность чисел не является псевдослучайной, но в результате ее зашифрования получается генератор случайных чисел, который можно использовать не только в криптографии.

Литература

- 1. Алгоритм криптографического преобразования ГОСТ 28147-89**
ИПК "Издательство стандартов", 1996
- 2. Data Encryption Standard (DES)**
Federal Information Processing Standards Publication 46-2
U.S. DEPARTMENT OF COMMERCE/National Institute of Standards and Technology
1993 December 30
- 3. Intel 80386 programmer's reference manual**
Intel, 1986
- 4. Cryptography and Computer Privacy**
Horst Feistel
Scientific American, May 1973, Vol. 228, No.5, pp. 15-23.
- 5. Алгоритм шифрования ГОСТ 28147-89, его использование и реализация для компьютеров платформы Intel x86**
Андрей Винокуров
"Монитор" ##1,5 за 1995 год
- 6. Как устроен блочный шифр?**
Андрей Винокуров
- 7. Защита информации в компьютерных системах и сетях**
В.Ф. Шаньгин
Радио и связь, 1999
- 8. Криптография от папируса до компьютера**
Владимир Жельников
ABF, 1996

Исходный текст функции зашифрования в режиме простой замены

```
; Последовательность выбора ключей
J1      db      0,1,2,3,4,5,6,7
        db      0,1,2,3,4,5,6,7
        db      0,1,2,3,4,5,6,7
        db      7,6,5,4,3,2,1,0

; Цикл 32-3 (32-P)
; Входные параметры:
; EDX = Указатель на ключ X
; ESI = N1
; EDI = N2
; Возвращаемые значения:
; EDX = Указатель на ключ X
; ESI = N1
; ESI = N2

_L32E   proc     near

        push     ecx
        mov      ebx,PS8                ; 4 8-битных узла замены
        xor      ecx,ecx                ; j := 0

; Цикл (j1)

_L32E_1:
        movzx    eax,J1[ecx]
        mov      eax,dword ptr [edx+eax*4] ; X[j1]
        add      eax,esi                ; CM1 := N1 + X[j1]

; Блок подстановки

        xlat
        add      ebx,100h
        ror      eax,8
        xlat
        add      ebx,100h
        ror      eax,8
        xlat
        add      ebx,100h
        ror      eax,8
        xlat
        sub      ebx,300h
        rol      eax,3
        xor      eax,edi                ; CM2 := R ^ N2

        mov      edi,esi                ; N2 := N1
        mov      esi,eax                ; N1 := CM2
        inc      ecx                    ; j := j + 1
        cmp      ecx,32                 ; j < 32 ?
        jb       _L32E_1
        xchg     esi,edi                ; N1 <-> N2
        pop      ecx
        ret

_L32E   endp
```

Исходный текст функции шифрования в режиме гаммирования

```
; Константы C1, C2 (заполнение N5, N6)
C1      equ      01010104h
C2      equ      01010101h

; Функция шифрования в режиме гаммирования
Параметры:
    @PBlock - указатель на блок данных
    @GSize  - размер блока данных (в байтах)
    @PKey   - указатель на ключ (256 бит)
    @PSync  - указатель на синхропосылку (64 бита)

CG32    proc      @PBlock:dword, @GSize:dword, @PKey:dword, @PSync:dword
local   @N3:dword, @N4:dword

        push     esi
        push     edi
        push     ebx
        push     ecx
        mov      ebx, @PSync
        mov      esi, [ebx]          ; N1 := S[0]
        mov      edi, [ebx+4]        ; N2 := S[1]
        mov      edx, @PKey
        call     _L32E
        mov      @N3, esi            ; N3 := N1
        mov      @N4, edi            ; N4 := N2
        mov      ecx, @GSize

CG32_Next:
        jecxz    CG32_End
        mov      eax, @N3
        add      eax, C2
        mov      @N3, eax
        mov      esi, eax            ; N1 := N3
        mov      eax, @N4
        add      eax, C1
        adc      eax, 0
        mov      @N4, eax
        mov      edi, eax            ; N2 := N4

; Зашифрование в режиме простой замены

        mov      edx, @PKey
        call     _L32E
        mov      ebx, @PBlock
        mov      eax, esi
        cmp      ecx, 4
        jb      CG32_Last
        xor      [ebx], eax
        add      ebx, 4
        sub      ecx, 4
        mov      eax, edi
        cmp      ecx, 4
        jb      CG32_Last
        xor      [ebx], eax
        add      ebx, 4
        sub      ecx, 4
        mov      @PBlock, ebx
        jmp      CG32_Next
```

```

CG32_Last:
    jecxz    CG32_End
    xor      [ebx],al
    inc      ebx
    ror      eax,8
    loop     CG32_Last
CG32_End:
    pop      ecx
    pop      ebx
    pop      edi
    pop      esi
    ret
CRG32      endp
end

```


Приложение 3

Аннотация

Исследование различных реализаций алгоритма криптографического преобразования
Рекомендации по построению систем криптографического преобразования
Определение дополнительных требований к узлам замен
Корреляционный анализ узлов замен ГОСТ 28147-89 и DES